

Bericht
Technik der Mensch-Maschine-Interaktion
SS 2025

Titel der Arbeit

erstellt von:
Abdi, Achraf
Sboui, Firas
Studiengang: Mechatronik

Kurzfassung

Im Rahmen dieses Projekts wurde ein interaktives Tic-Tac-Toe-Spiel mit dem humanoiden Roboter NAO entwickelt. Ziel war es, ein System zu schaffen, bei dem der Roboter ein physisches Spielfeld mithilfe von visuellen Markern analysiert, den Spielzustand intern verarbeitet und seine Spielzüge durch Sprachausgabe kommuniziert. Der Mensch übernimmt dabei sowohl die Rolle des Gegenspielers als auch die physische Umsetzung der vom Roboter vorgegebenen Züge.

Technisch basiert das System auf einer Kombination aus NAOs integrierter Kamera, Choreographie-Software und Python-Skripten. Statt Spracheingabe erfolgt die Interaktion einseitig: NAO beobachtet und spricht – der Benutzer handelt entsprechend. Die Markererkennung erfolgt über ID-Zuordnung (leer, X, O), die Spiellogik über eine interne 3×3-Matrix.

Das Projekt zeigt, wie verschiedene Elemente der Robotik – Bildverarbeitung, Bewegungssteuerung und Mensch-Roboter-Interaktion – in einer konkreten Anwendung erfolgreich verbunden werden können.

Inhaltsverzeichnis

1	Einleitung	5
1.1	Persönliche Motivation und Zielsetzung	5
1.2	Projektidee und Hintergrund	5
1.3	Methodisches Vorgehen und Teamarbeit	5
2	Projektentwicklung und Lösungsweg	6
2.1	Ursprünglicher Ansatz: Objekterkennung von X und O	6
2.2	Markerbasierte Spielfeldererkennung mit Einzelzuordnung	6
2.3	Optimiertes Markerprinzip mit Symbolzuweisung	6
2.4	Steuerung des Spielablaufs über NAO-Ausgabe	6
2.5	Optimierung der Kameraeinstellungen zur Markeranalyse	7
3	Technische Grundlagen	7
3.1	Überblick: NAO-Roboter und Sensorik	7
3.2	Software: Choregraphie und NAOqi	7
3.3	Markererkennung mit integrierter Kamera	8
3.4	Sprachsynthese und Mensch-Roboter-Dialog	8
4	Systemdesign und Spielmechanik	8
4.1	Spielfeldstruktur und Markerzuordnung	8
4.2	Interne Gitterinitialisierung und Statusspeicherung	9
4.3	Ablauflogik: Spielstart, Zugfolge und Spielende	9
4.4	Bewegungsabläufe des Roboters während der Analyse	10
4.5	Benutzerinteraktion und Rollenverteilung	10
5	Softwarestruktur und Implementierung	12
5.1	Aufbau des Choregraphie-Projekts	12
5.2	Python-Skripte zur Markerinterpretation und Spiellogik	13
5.3	Beispielhafte Codeabschnitte mit Kommentierung	22
6	Entwicklungsprobleme und Lösungen	23
6.1	Marker-ID-Probleme und Fehlinterpretationen	23

6.2	Einschränkungen bei der Kameraperspektive.....	24
6.3	Schwierigkeit der Bewegungskalibrierung.....	24
6.4	Optimierung durch manuelle Testzyklen	24
7	Fazit und Ausblick	25
7.1	Bewertung der Projektergebnisse	25
7.2	Reflexion des Entwicklungsprozesses	25
7.3	Erweiterungsmöglichkeiten (z. B. Spracherkennung, AI-Gegner)	25
8	Literaturverzeichnis	26

1 Einleitung

1.1 Persönliche Motivation und Zielsetzung

Die Faszination für humanoide Roboter und deren Möglichkeiten zur Interaktion mit der physischen Umwelt war der Ausgangspunkt für dieses Projekt. Als interdisziplinäres Team von Mechatronik-Studierenden war es unser Ziel, ein reales Anwendungsbeispiel zu schaffen, das Robotik, Wahrnehmung, Bewegung und logische Entscheidungsfindung miteinander verbindet.

Das klassische Spiel Tic-Tac-Toe wurde bewusst gewählt, da es sich durch eine einfache Struktur und klare Regeln auszeichnet, gleichzeitig aber eine Vielzahl technischer Herausforderungen mit sich bringt, wenn es auf reale Interaktion und Robotik übertragen wird. Ziel war es, ein System zu entwickeln, bei dem der Roboter selbstständig das Spielfeld analysiert, Spielzüge verarbeitet und verbal kommuniziert, welche Aktionen der Benutzer durchführen soll. Sprachsteuerung war dabei nicht Bestandteil des Projekts; die Kommunikation verlief einseitig vom Roboter zum Menschen.

1.2 Projektidee und Hintergrund

Das Spiel basiert auf einem physischen Spielfeld mit visuellen Markern, das dem Roboter als Eingabefläche dient. Die Marker sind so kodiert, dass sie verschiedene Spielzustände repräsentieren (leer, X oder O). Der Roboter überprüft durch fest definierte Kopfbewegungen nacheinander die neun Felder, erkennt anhand der Marker den aktuellen Zustand und aktualisiert intern eine digitale Gittermatrix.

Anschließend berechnet er seinen eigenen Spielzug und gibt diesen per Sprache aus. Der menschliche Benutzer übernimmt die Aufgabe, die Anweisungen des Roboters auf dem Spielfeld umzusetzen, wodurch eine reaktive Mensch-Roboter-Interaktion entsteht. Die Herausforderung bestand darin, die Marker sicher zu erkennen, den Spielfluss stabil zu halten und die Erkennung visuell und logisch konsistent umzusetzen.

1.3 Methodisches Vorgehen und Teamarbeit

Das Projekt wurde in mehreren Iterationen entwickelt. Ausgangspunkt war eine grobe Spielidee, die über verschiedene technische Ansätze verfeinert wurde. In jeder Phase testeten wir neue Möglichkeiten zur Felderkennung, verbesserten die Markerstruktur und kalibrierten die Bewegungs- und Scanabläufe.

Die Arbeit im Team war dabei entscheidend für den Projekterfolg. Aufgaben wurden aufgeteilt, Lösungen gemeinsam evaluiert und technische Probleme im Austausch gelöst. Neben der technischen Umsetzung ermöglichte das Projekt auch die Vertiefung wichtiger Soft Skills wie Kommunikation, Problemlösung, Dokumentation und iterative Entwicklung.

Die folgenden Kapitel dokumentieren den Entwicklungsweg, die eingesetzten Technologien sowie Herausforderungen und Ergebnisse des Projekts.

2 Projektentwicklung und Lösungsweg

2.1 Ursprünglicher Ansatz: Objekterkennung von X und O

Der erste technische Ansatz zielte darauf ab, dass der NAO-Roboter auf einem leeren Spielfeld handschriftlich gesetzte X- und O-Symbole erkennen kann. Dies sollte mithilfe seiner integrierten Kamera geschehen. Die Idee war, dass NAO durch Formunterscheidung visuell analysiert, ob ein bestimmtes Feld leer, mit einem X oder einem O besetzt ist.

Dieser Ansatz scheiterte an der praktischen Umsetzung. Die Auflösung der Kamera, fehlende Trainingsdaten und die fehlende Fähigkeit zur freien Bilderkennung ohne KI-Unterstützung führten dazu, dass NAO die Zeichen nicht zuverlässig unterscheiden konnte. Auch der Versuch, dem System zahlreiche Varianten von X und O beizubringen, blieb ohne Erfolg.

2.2 Markerbasierte Spielfeldererkennung mit Einzelzuordnung

Im nächsten Schritt versuchten wir, jedem Feld des 3×3-Gitters einen individuellen Marker zuzuweisen. Damit hätte NAO jedes Feld eindeutig identifizieren und dessen Zustand erfassen können. Die Umsetzung erfolgte mit NAO-Markern, die jeweils eine eindeutige ID repräsentierten.

Die Grundidee funktionierte: NAO konnte die Marker erkennen und ihnen logische Feldpositionen zuordnen. Allerdings stießen wir auf ein praktisches Problem: Die gleichzeitige Erkennung von neun unterschiedlichen Markern auf einem A3-Blatt erwies sich als unzuverlässig. Marker wurden durch Licht, Schatten oder Perspektive übersehen. Das System war dadurch zu fehleranfällig.

2.3 Optimiertes Markerprinzip mit Symbolzuweisung

Aus den zuvor gesammelten Erkenntnissen entwickelten wir ein vereinfachtes System. Statt jedem Feld einen eigenen Marker zuzuweisen, entschieden wir uns für eine Symbol-basierte Markerlogik:

ID 84 = Feld ist leer

ID 64 = Feld enthält ein "X"

ID 112 = Feld enthält ein "O"

Jedes Feld auf dem Spielfeld wurde somit zu Beginn mit einem Marker der ID 84 versehen. Sobald der Benutzer einen Spielzug durchführt, wird der Marker durch ID 64 oder 112 ersetzt. NAO erkennt beim Scannen nur die Marker-ID und schließt daraus auf den Spielzustand. Die genaue Feldposition wird durch die Reihenfolge der Scanbewegung definiert.*

2.4 Steuerung des Spielablaufs über NAO-Ausgabe

Der Ablauf des Spiels beginnt mit dem Start über Choregraphie. Der Roboter führt systematisch neun Kopfbewegungen aus, um alle Spielfelder visuell zu erfassen. Dabei interpretiert er die Marker-IDs, initialisiert sein internes Spielfeld und beschreibt den Zustand per Sprache.

Nach der Initialisierung entscheidet NAO anhand seiner Spielstrategie, welches Feld er als nächstes belegen will. Er spricht seinen gewählten Spielzug aus, und der Benutzer platziert dann den entsprechenden Marker (O) auf dem physischen Feld.

2.5 Optimierung der Kameraeinstellungen zur Markeranalyse

Ein wesentlicher Erfolgsfaktor war die visuelle Optimierung. Mithilfe des Choregraphe-Monitors konnten wir Bildauflösung, Kontrast, Helligkeit und Belichtungszeit manuell anpassen. Durch gezielte Tests fanden wir Parameterkombinationen, bei denen Marker auch bei unterschiedlichen Lichtverhältnissen sicher erkannt wurden.

Diese Optimierungen reduzierten das Risiko von Fehldetektionen (z. B. ID -1) erheblich und machten das System robuster für den praktischen Einsatz.

3 Technische Grundlagen

3.1 Überblick: NAO-Roboter und Sensorik

Der NAO-Roboter wurde ursprünglich von der französischen Firma Aldebaran Robotics entwickelt und wird heute von SoftBank Robotics weitergeführt und vertrieben. Es handelt sich um einen humanoiden Roboter, der speziell für Forschungs- und Bildungszwecke konzipiert wurde., der speziell für Forschungs- und Bildungszwecke entwickelt wurde. Er verfügt über zahlreiche Sensoren, darunter zwei HD-Kameras, mehrere Mikrofone, Infrarot-Sensoren, Sonar sowie Berührungssensoren an Kopf, Händen und Füßen. Die integrierten Kameras im Stirn- und Mundbereich ermöglichen die visuelle Erfassung von Objekten und Markern im Sichtfeld.

Für dieses Projekt wurden sowohl die obere als auch die untere Kamera zur Markererkennung verwendet, um je nach Blickwinkel und Feldposition eine optimale Erkennung zu gewährleisten. Die Kopfbeweglichkeit des Roboters ermöglicht das gezielte Anfahren einzelner Spielfelder durch horizontale und vertikale Rotation, was für die systematische Analyse des Spielfelds erforderlich war.

3.2 Software: Choregraphe und NAOqi

Die Programmierung und Steuerung des Roboters erfolgte über die grafische Entwicklungsumgebung Choregraphe in Kombination mit der Programmiersprache Python.

Choregraphe bietet eine Block-basierte Struktur, in der Bewegungen, Logik, Sensorik und Sprachmodule modular miteinander verknüpft werden können. Zusätzlich ermöglicht Choregraphe die direkte Einbindung von Python-Skripten über sogenannte "Python Script"-Boxen. Diese wurden genutzt, um die Marker zu interpretieren, die Spiellogik umzusetzen und die Gitterstruktur zu aktualisieren.

Die Steuerung basiert auf dem Software-Framework NAOqi, das alle Roboterfunktionen abstrahiert. Es erlaubt u. a. den Zugriff auf Bewegungs-APIs, die Sprachausgabe, die Speicherverwaltung (ALMemory) sowie Kamera-Streams und Bildverarbeitung.

3.3 Markererkennung mit integrierter Kamera

Zur Erkennung der Spielzüge nutzt NAO sogenannte NAOMarkers. Diese bestehen aus schwarz-weißen Quadraten, die eine eindeutige ID codieren. Die Markererkennung erfolgt über die ALVisionRecognition-Komponente der NAOqi-API. Die Marker müssen im Sichtfeld zentriert, ausreichend beleuchtet und in einer geeigneten Größe dargestellt werden.

Für dieses Projekt wurden drei Marker-IDs verwendet: ID 84 = leer | ID 64 = X | ID 112 = O

Die Marker wurden auf Papier gedruckt und mittig in jedem Spielfeld platziert. Der Roboter bewegte seinen Kopf sequentiell über die neun Felder, analysierte das aktuelle Markerbild, verglich die ID und überführte das Ergebnis in ein digitales Spielfeld (3x3-Matrix).

3.4 Sprachsynthese und Mensch-Roboter-Dialog

Die Interaktion zwischen Roboter und Spieler erfolgt ausschließlich über Sprachausgabe. NAO nutzt die integrierte Komponente ALTextToSpeech, um Texte in gesprochene Sprache umzuwandeln.

Diese Funktion wurde genutzt, um den Benutzer über den Spielstatus zu informieren, Anweisungen zu geben (z. B. "Bitte lege mein O auf Mitte rechts") und Entscheidungen zu erklären. Eine Spracherkennung kam nicht zum Einsatz; der Benutzer hört die Ausgaben und handelt dementsprechend.

Die Sprachmodule wurden direkt in Choregraphie hinterlegt und an zentrale Zustände der Spiellogik gekoppelt, sodass jede Entscheidung eine eindeutige verbale Reaktion auslöste. Durch die sprachliche Interaktion wurde ein intuitives Spielgefühl erzeugt, obwohl keine vollständige Dialogfähigkeit realisiert wurde.

4 Systemdesign und Spielmechanik

4.1 Spielfeldstruktur und Markerzuordnung

Das Spielfeld besteht aus neun Feldern in einem 3x3-Raster, auf denen NAOMarker eingesetzt werden. Ursprünglich war angedacht, jedes Feld mit einem individuellen Marker zu versehen, was jedoch zu Erkennungsproblemen führte. Die finale Lösung war es, nur drei Marker-IDs zu verwenden:

84 = leer

64 = X

112 = O

Diese Marker wurden in jedes Feld je nach Inhalt platziert, was den Erkennungsprozess vereinfachte und standardisierte.



Abb1: Gitter

4.2 Interne Gitterinitialisierung und Statusspeicherung

Nach dem Start initialisiert NAO ein internes Gitter mit dem Zustand „leer“. Bei jedem Scan wird der Status (X, O, leer) aus dem Markerwert abgeleitet und in einer Matrix gespeichert. Diese Struktur ist die Grundlage für die Spiellogik und wird im LogViewer ausgegeben.

In der Simulation erfolgt die Initialisierung und Markerinterpretation stabil, da dort die Markerwerte direkt simuliert werden. In der realen Umgebung jedoch hängt die erfolgreiche Initialisierung stark von der korrekten Kameraposition und der Lesbarkeit der Marker ab. Eine falsche Perspektive oder ungenaue Beleuchtung kann bereits dazu führen, dass das Feld nicht korrekt erkannt wird. Deshalb wurde später zusätzlich eine sprachbasierte Rückfragefunktion für unklare Markerfälle ergänzt (siehe Kapitel 6.1).

4.3 Ablauflogik: Spielstart, Zugfolge und Spielende

Das Spiel beginnt mit dem Befehl aus Choregraphie. NAO startet die Analyse aller neun Felder durch Kopfbewegungen. Danach sagt er per Sprachausgabe seinen Spielzug an. Der Mensch setzt den Spielstein auf das entsprechende Feld und bestätigt über Kopfberührung. Danach beginnt der nächste Zyklus.

4.4 Bewegungsabläufe des Roboters während der Analyse

Der Kopf des Roboters bewegt sich sequenziell über alle neun Felder. Diese Bewegungen wurden in Choreographie als feste Bewegungsabfolgen programmiert. Dabei wird je nach Feld entweder die obere oder untere Kamera verwendet – abhängig von Sichtfeld und Winkel.

Während in der Simulation die Bewegungen des Roboters klar und deterministisch ablaufen, traten im Realbetrieb bei echten Bewegungsabläufen zusätzliche Herausforderungen auf. Dazu gehörten Unsicherheiten bei der Kameraausrichtung, schwankende Marker-Erkennung durch Lageabweichungen sowie eine stärkere Anfälligkeit bei Randfeldern des Gitters. Diese Effekte mussten durch zusätzliche Optimierungen der Bewegungslogik und Testwiederholungen abgefedert werden.



Abb2: NAO-Roboter beim Scannen der Spielfeld-Marker in Crouch-Position

4.5 Benutzerinteraktion und Rollenverteilung

Die Interaktion ist einseitig: NAO spricht, der Mensch hört zu und handelt. Der Benutzer bestätigt NAOs Zug per Kopfberührung. Diese klare Rollenverteilung vereinfacht das Spiel.

Die Bewegungsabfolgen, Kopfrotationen und Kameraeinstellungen wurden in zahlreichen Testläufen iterativ verfeinert. Helligkeit, Kontrast, Sättigung und Framerate wurden dabei über den Monitor (Monitor Desktop) justiert, um die Markererkennung zu verbessern. In der verwendeten Einstellung erwies sich die Kombination aus:

Auflösung: 640 × 480

Kontrast: 39

Helligkeit: 94

Sättigung: 121

als besonders robust für unterschiedliche Lichtverhältnisse.

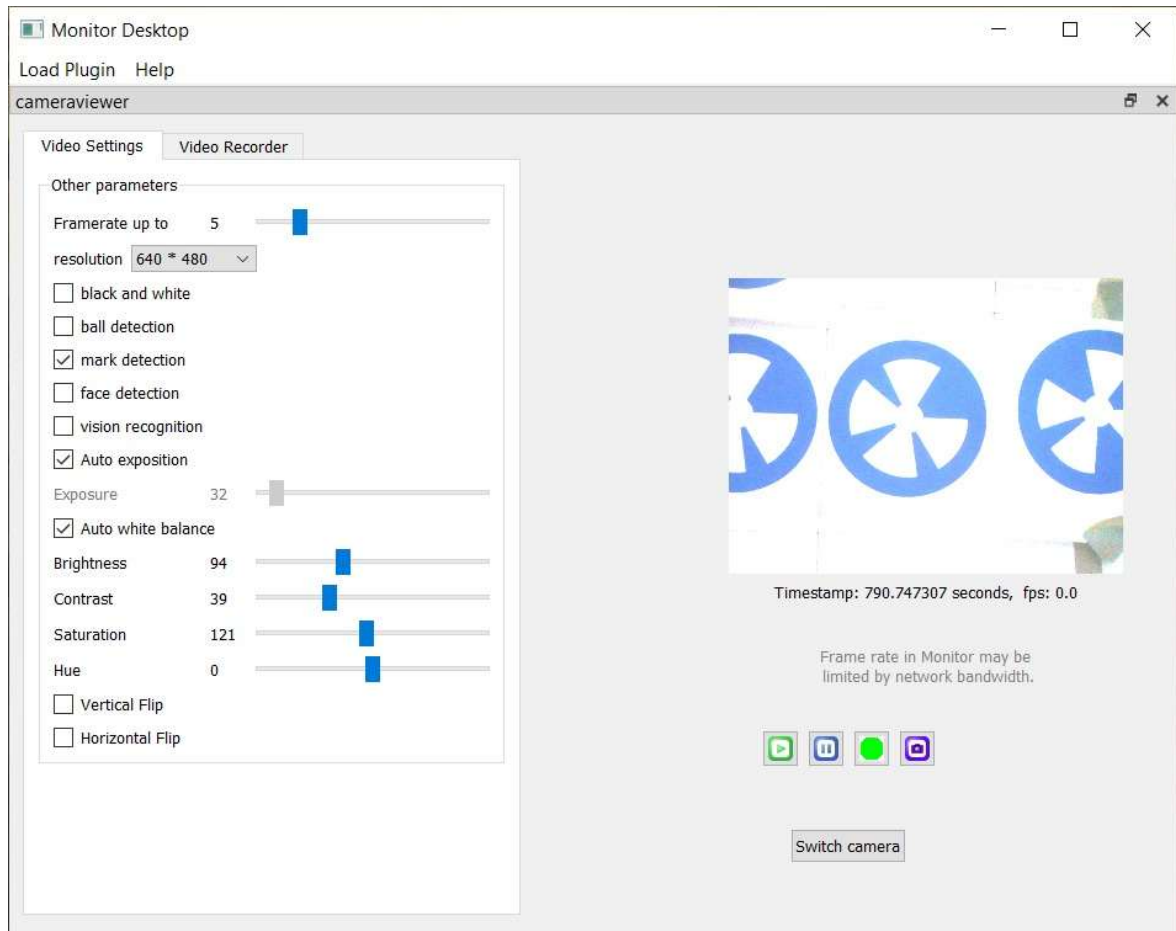


Abb3: Monitor Desktop

Es ist wichtig zu betonen, dass das gesamte System in der Choregraphie-Simulation äußerst zuverlässig funktioniert: Die Marker-IDs werden dort nicht über reale Kamerabilder erkannt, sondern direkt simuliert, wodurch alle Spielzüge korrekt verarbeitet werden. Bewegungen, Spielstatus und Ablauf laufen stabil und ohne Unterbrechungen ab.

In der realen Ausführung mit dem physischen NAO-Roboter traten jedoch Einschränkungen auf. Insbesondere die Marker-ID-Erkennung war teilweise fehlerhaft (z. B. ID -1), was an folgenden Faktoren lag:

leichte Abweichungen zwischen Spielfeld und Kameraperspektive

begrenzttes Sichtfeld der Kamera

optische Störungen durch Beleuchtung oder Kamerawinkel

Diese Probleme führten dazu, dass Marker falsch erkannt oder übersehen wurden. Die Ursache liegt weniger im Code, sondern vielmehr in den physikalischen Begrenzungen der NAO-Hardware und der Notwendigkeit, Marker möglichst ideal im Sichtfeld zu positionieren.

Besondere Bedeutung kam der Benutzerrolle bei der manuellen Korrektur von Erkennungsfehlern zu: Wenn der Roboter ein Feld nicht eindeutig erkennen konnte, wurde der Nutzer per Sprache dazu aufgefordert, den Zustand des Feldes (leer, X oder O) zu nennen. Diese Form der Interaktion machte das System robuster gegenüber Hardwaregrenzen.

In der Simulation jedoch, bei identischen Softwarekomponenten, funktionierte das System fehlerfrei.

5 Softwarestruktur und Implementierung

5.1 Aufbau des Choregraphie-Projekts

Die gesamte Spiellogik und Steuerung wurde mithilfe der grafischen Programmierumgebung Choregraphie umgesetzt. Das zentrale Projekt bestand aus mehreren Blöcken (Boxen), welche die Hauptfunktionen kapseln:

Tic_Tac_Toe Box (Python-Skript mit Spiellogik und Sprachausgabe)

Scanner Box (Startsignal zur Markeranalyse)

Marker Box (verarbeitet erkannte Marker und gibt Werte weiter)

Die nachfolgende Abbildung zeigt die visuelle Struktur des Projekts in Choregraphie:

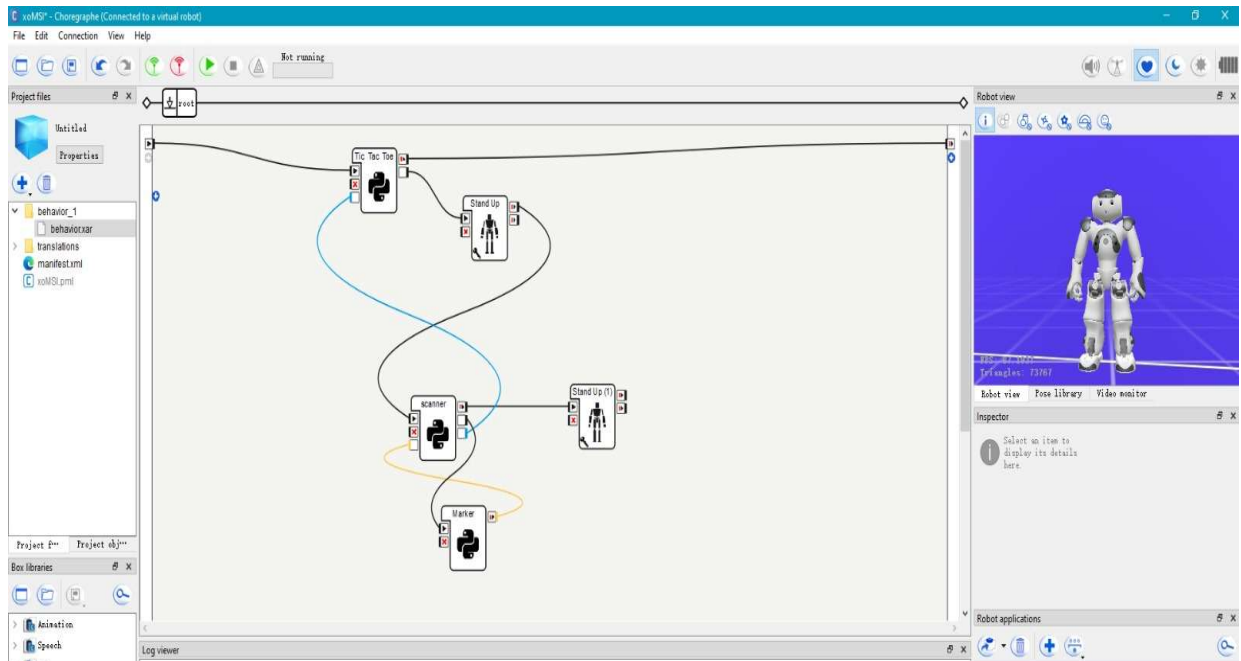


Abb4: Übersicht des Choregraphe-Verhaltensflusses für das Tic-Tac-Toe-Spiel

5.2 Python-Skripte zur Markerinterpretation und Spiellogik

Die **Tic_Tac_Toe Box** bildet den Kern der Spiellogik. Sie enthält ein Python-Skript, das folgende Aufgaben übernimmt:

Initialisierung des Spielfelds (3×3 Matrix)

Annahme von Markerwerten („X“, „O“, „leer“)

Kontrolle auf Sieg oder Unentschieden

Sprachausgabe für jeden Spielzug

Auslösung neuer Scans durch die Scanner-Box

Im Folgenden wird der vollständige Python-Code gezeigt:

```
1 from naoqi import ALProxy
2 import time
3
4 class MyClass(GeneratedClass):
5     def __init__(self):
6         GeneratedClass.__init__(self)
7         self.grid = [["leer" for _ in range(3)] for _ in range(3)] # Initialisiere das 3x3
Gitter mit "leer"
8         self.current_index = 0
9         self.zug = 0
10        self.memory = None
11        self.logGrid("Zug [0]: Gitter wurde initialisiert")
12
13    def onLoad(self):
14        self.tts = ALProxy("ALTextToSpeech")
15        self.memory = ALProxy("ALMemory")
16
17
18
19    def onUnload(self):
20        pass
21
22    def onInput_onStart(self):
23
24
25        self.sendScanRequest()
26
27    def onInput_onStop(self):
28        self.onUnload()
29
30    def onInput_fieldValue(self, value):
31        if self.current_index >= 9:
32            self.logger.warning("Mehr als 9 Eingaben erhalten. Ignoriere weiteren Wert.")
33            return
34
35        row = self.current_index // 3
36        col = self.current_index % 3
37
38        if value in ["X", "O", "leer"]:
39            self.grid[row][col] = value
40            self.current_index += 1
41        else:
42            self.logger.warning("Ungültiger Feldwert empfangen: " + value)
43
44        if self.current_index == 9:
45            self.zug += 1
46            self.logGrid("Zug [{}].format(self.zug))
47
48            if self.checkWinner():
49                return
50
51            if self.checkDraw():
52                return
53
54            self.robotMove()
55
56
57    def sendScanRequest(self):
58        try:
59            #self.logger.info("Scanner wird ausgelöst...")
60            self.ScanRequest()
61        except Exception:
62            self.logger.error("Scanner-Auslösung fehlgeschlagen.")
63
64
65    def onOutput_ScanRequest(self):
66        pass
67
```



```

68
69
70 def logGrid(self, header):
71     separator = "-----"
72     rows = ["| {} | {} | {} |".format(*row) for row in self.grid]
73     grid_string = separator.join(rows)
74     self.logger.info("{}: {}".format(header, grid_string))
75
76 def robotMove(self):
77     empty_cells = [(i, j) for i in range(3) for j in range(3) if self.grid[i][j] == "leer"]
78     if not empty_cells:
79         return
80
81     row, col = empty_cells[0]
82     self.grid[row][col] = "0"
83     feld_nummer = row * 3 + col
84     self.tts.say("Ich setze ein 0 auf Feld " + str(feld_nummer))
85     self.logger.info("Roboter setzt 0 auf Feld {}".format(feld_nummer))
86
87     self.tts.say("Bitte machen Sie Ihren Zug und drücken Sie meinen Kopf.")
88
89     # Warten bis Kopf gedrückt wird
90     while True:
91         try:
92             if self.memory.getData("FrontTactilTouched") == 1.0:
93                 break
94         except:
95             pass
96         time.sleep(0.2)
97
98
99     self.current_index = 0 #index reinitialisiert für nächstes Scannen
100     #self.logger.warning("Currentindex wurde initialisiert")
101
102     self.sendScanRequest() #neu Feld Scannen wird ausgelöst
103
104
105 def checkWinner(self):
106     # Zeilen prüfen
107     for row in self.grid:
108         if row == ["X", "X", "X"]:
109             self.logger.info("Spieler hat gewonnen")
110             self.tts.say("Du hast gewonnen!")
111             self.onStopped()
112             return True
113         if row == ["0", "0", "0"]:
114             self.logger.info("Nao hat gewonnen")
115             self.tts.say("Ich habe gewonnen!")
116             self.onStopped()
117             return True
118
119     # Spalten prüfen
120     for col in range(3):
121         if self.grid[0][col] == self.grid[1][col] == self.grid[2][col] != "leer":
122             winner = self.grid[0][col]
123             self.tts.say("Du hast gewonnen!" if winner == "X" else "Ich habe gewonnen!")
124             self.onStopped()
125             return True
126
127     # Diagonale (links oben → rechts unten)
128     if self.grid[0][0] == self.grid[1][1] == self.grid[2][2] != "leer":
129         winner = self.grid[0][0]
130         self.tts.say("Du hast gewonnen!" if winner == "X" else "Ich habe gewonnen!")
131         self.onStopped()
132         return True
133
134     # Diagonale (rechts oben → links unten)
135     if self.grid[0][2] == self.grid[1][1] == self.grid[2][0] != "leer":
136

```

```
137         winner = self.grid[0][2]
138         self.tts.say("Du hast gewonnen!" if winner == "X" else "Ich habe gewonnen!")
139         self.onStopped()
140         return True
141
142     return False
143
144
145     def checkDraw(self):
146         if all(cell != "leer" for row in self.grid for cell in row):
147             self.tts.say("Unentschieden!")
148             self.logger.info("Unentschieden!")
149             self.onStopped()
150             return True
151         return False
```

Die Kommunikation erfolgt sequentiell: Nach jedem Spielzug wird auf die Kopfberührung gewartet, bevor der nächste Scan ausgelöst wird.

Die **Scanner Box** ist für die Auslösung des Scan-Vorgangs verantwortlich. Sie triggert ein benutzerdefiniertes Ereignis, das NAO anweist, die aktuelle Kameraposition zu analysieren. Der Trigger wird am Anfang des Spiels sowie nach jeder Kopfberührung erneut aktiviert. In der Scanner-Box wird keine komplexe Logik ausgeführt – sie dient als Startsignal für die Markerauswertung.

Im Folgenden wird der vollständige Python-Code gezeigt:


```
1 from naoqi import ALProxy
2 import time
3
4 class MyClass(GeneratedClass):
5     def __init__(self):
6         GeneratedClass.__init__(self)
7         self.current_field = (0, 0)
8         self.tts = None
9         self.camera = None
10
11     def onLoad(self):
12         self.tts = ALProxy("ALTextToSpeech")
13         self.camera = ALProxy("ALVideoDevice")
14
15     def wait_seconds(self, seconds):
16
17         time.sleep(seconds)
18
19     def wait_for_rear_touch(self):
20         self.tts.say("Bitte drücken Sie meinen Hinterkopf, um fortzufahren.")
21         while True:
22             try:
23                 if ALProxy("ALMemory").getData("RearTactilTouched") == 1.0:
24                     self.tts.say("Danke!")
25                     break
26             except:
27                 pass
28             time.sleep(0.2)
29
30     def warteAufMarkerEmpfang(self):
31         for _ in range(500):
32             if self.marker_empfangen:
33
34                 break
35             time.sleep(0.1)
36         else:
37             self.logger.info("50 Sekunden vergangen, kein Marker empfangen.")
38
39
40     def move_head(self, motion, yaw, pitch):
41         motion.setAngles(["HeadYaw", "HeadPitch"], [yaw, pitch], 0.15)
42         motion.waitUntilMoveIsFinished()
43         self.wait_seconds(1.0)
44
45
46     def interpret_marker(self, marker_id):
47         #self.logger.info("Interpretation mit ID aufgerufen ".format(marker_id))
48
49         if str(marker_id) == "84":
50             return "leer"
51         elif str(marker_id) == "64":
52             return "X"
53         elif str(marker_id) == "112":
54             return "0"
55         else:
56             # Unbekannte ID → frage den Spieler
57             self.tts.say("Ich habe ein Problem, dieses Feld zu interpretieren. Ist es ein X, ein
0 oder leer hier?")
58
59         # Spracheingabe aktivieren
60         try:
61             asr = ALProxy("ALSpeechRecognition")
62             asr.pause(True)
63             asr.setLanguage("German")
64             vocabulary = ["X", "0", "leer"]
65             asr.setVocabulary(vocabulary, False)
66             asr.subscribe("MarkerFallback")
67
```

```

68         self.logger.info("Warte auf Spracheingabe...")
69
70         memory = ALProxy("ALMemory")
71         result = None
72
73         # Warte max. 30 Sekunden
74         start_time = time.time()
75         while time.time() - start_time < 30:
76             val = memory.getData("WordRecognized")
77             if isinstance(val, list) and len(val) > 1:
78                 word = val[0]
79                 confidence = val[1]
80                 if word in vocabulary and confidence > 0.1:
81                     result = word
82                     break
83             time.sleep(0.1)
84
85         asr.unsubscribe("MarkerFallback")
86
87         if result:
88             self.tts.say("Danke, ich habe {} verstanden.".format(result))
89             return result
90         else:
91             self.tts.say("Keine klare Antwort erhalten. Ich nehme leer an.")
92             return "leer"
93
94         except Exception as e:
95             self.logger.error("Fehler bei Spracheingabe: " + str(e))
96             self.tts.say("Spracheingabe nicht verfügbar. Ich nehme leer an.")
97             return "leer"
98
99
100     def onInput_onMarkerResult(self, marker_id):
101
102         self.logger.info("Marker empfangen: {}".format(marker_id))
103
104         result = self.interpret_marker(marker_id) #Interpretation sofort nach dem Empfang
105
106         y, x = self.current_field #Feld position
107
108
109
110
111         try:
112             self.onFieldValue(result) #Interpretation an Tic_Tac_toe box senden
113             #self.logger.info(">> Wert '{}' an Gitter-Box gesendet.".format(result))
114         except Exception as e:
115             self.logger.error("Fehler beim Senden des Feldwerts: " + str(e))
116
117         self.marker_empfangen = True # Warten auf MarkerResult wird erst hier beendet,
118                                     # damit genügend Zeit für Interpretation und Sprachausgabe
119                                     # und der Roboter sich nicht zu früh bewegt.
120
121     def onInput_onStart(self):
122         motion = ALProxy("ALMotion")
123         posture = ALProxy("ALRobotPosture")
124
125         motion.setStiffnesses("Head", 1.0)
126         self.camera.setActiveCamera(0)
127         self.move_head(motion, 0, -0.1)
128         self.wait_seconds(1.0)
129
130         self.wait_for_rear_touch() #Warten bis tactilSensor gedrückt werden
131
132         #####Obere Reihe #####
133
134         #links
135         self.move_head(motion, -0.4, 0)

```

```
136         self.current_field = (0, 0) #Feld 0
137         self.wait_seconds(5.0)
138
139         self.marker_empfangen = False
140         self.onMarkRequest()
141         self.warteAufMarkerEmpfang()
142
143
144         #Mitte
145         self.move_head(motion, 0, 0)
146         self.current_field = (0, 1) #Feld 1
147         self.wait_seconds(5.0)
148
149         self.marker_empfangen = False
150         self.onMarkRequest()
151         self.warteAufMarkerEmpfang()
152
153
154         #Rechts
155         self.move_head(motion, 0.3, 0)
156         self.current_field = (0, 2) #Feld 2
157         self.wait_seconds(5.0)
158
159         self.marker_empfangen = False
160         self.onMarkRequest()
161         self.warteAufMarkerEmpfang()
162
163
164         ##### Mittlere Reihe #####
165
166         #links
167         posture.goToPosture("Crouch", 0.6)
168         self.move_head(motion, -0.4, 0.1)
169         self.current_field = (1, 0) #Feld 3
170         self.wait_seconds(5.0)
171
172         self.wait_for_rear_touch() #Warten bis tactilSensor gedrückt werden
173
174         motion.setStiffnesses("Legs", 0.0) # Damit die Beinmotoren entlastet und vor Überhitzung
geschützt werden
175
176         self.marker_empfangen = False
177         self.onMarkRequest()
178         self.warteAufMarkerEmpfang()
179
180
181         ##Mitte
182         self.move_head(motion, 0, 0.1)
183         self.current_field = (1, 1) #Feld 4
184         self.wait_seconds(5.0)
185
186         self.marker_empfangen = False
187         self.onMarkRequest()
188         self.warteAufMarkerEmpfang()
189
190
191         #Rechts
192         self.move_head(motion, 0.3, 0.1)
193         self.current_field = (1, 2) #Feld 5
194         self.wait_seconds(5.0)
195
196         self.marker_empfangen = False
197         self.onMarkRequest()
198         self.warteAufMarkerEmpfang()
199
200
201         ##### Untere Reihe#####
202
203         #links
```

```
204         self.camera.setActiveCamera(1)
205         self.move_head(motion, -0.4, -0.2)
206         self.current_field = (2, 0) #Feld 6
207         self.wait_seconds(5.0)
208
209         self.marker_empfangen = False
210         self.onMarkRequest()
211         self.warteAufMarkerEmpfang()
212
213
214         #Mitte
215         self.move_head(motion, 0, -0.2)
216         self.current_field = (2, 1) #Feld 7
217         self.wait_seconds(5.0)
218
219         self.marker_empfangen = False
220         self.onMarkRequest()
221         self.warteAufMarkerEmpfang()
222
223
224         #Rechts
225         self.move_head(motion, 0.4, -0.2)
226         self.current_field = (2, 2) #Feld 8
227         self.wait_seconds(5.0)
228
229         self.marker_empfangen = False
230         self.onMarkRequest()
231         self.warteAufMarkerEmpfang()
232
233
234         motion.setStiffnesses("Legs", 1.0) # Steifigkeit der Beine wiederherstellen, damit
Bewegungen möglich sind
235         self.logger.info("Scan abgeschlossen. Ergebnis:")
236
237         self.onStopped()
238         self.camera.setActiveCamera(1)
239
240     def onInput_onStop(self):
241         self.onStopped()
```

Die **Marker Box** empfängt das Bildsignal der Kamera, liest die erkannte Marker-ID aus und leitet den Wert weiter an die Tic_Tac_Toe-Box. Je nach erkannter ID (84, 64 oder 112) wird ein Feld im Gitter als "leer", "X" oder "O" eingetragen. Die Marker Box nutzt eine einfache If-Else-Struktur zur Interpretation und Weiterleitung der Werte.

Im Folgenden wird der vollständige Python-Code gezeigt:

```

1  from naoqi import ALProxy
2  import time
3
4  class MyClass(GeneratedClass):
5      def __init__(self):
6          GeneratedClass.__init__(self, False)
7          self.landmark_proxy = None
8          self.memory = None
9          self.tts = None
10         self.camera = None
11
12     def onLoad(self):
13         try:
14             self.landmark_proxy = ALProxy("ALLandMarkDetection")
15             self.memory = ALProxy("ALMemory")
16             self.tts = ALProxy("ALTextToSpeech")
17             self.camera = ALProxy("ALVideoDevice")
18
19             self.landmark_proxy.subscribe("MarkerBox")
20         except Exception as e:
21             self.logger.error("Fehler bei Initialisierung: " + str(e))
22
23     def onUnload(self):
24         try:
25             self.landmark_proxy.unsubscribe("MarkerBox")
26         except:
27             pass
28
29     def onInput_onStart(self):
30         marker_id = -1 # Default
31
32         # Clear old landmark data
33         try:
34             self.memory.removeData("LandmarkDetected")
35         except:
36             pass
37
38         #self.tts.say("Suche nach Marker...")
39
40         # Try to detect marker for up to 5 seconds
41         start_time = time.time()
42         timeout = 5 # seconds
43
44         while time.time() - start_time < timeout:
45             try:
46                 data = self.memory.getData("LandmarkDetected")
47                 if isinstance(data, list) and len(data) > 1:
48                     marks = data[1]
49                     if len(marks) > 0:
50                         marker_id = marks[0][1][0] # Get first marker ID
51                         #self.tts.say("Marker erkannt. ID: " + str(marker_id))
52                         self.onStopped(marker_id) # Stop immediately with result
53                         return
54             except:
55                 pass
56
57             time.sleep(0.2)
58
59         # Timeout fallback
60         self.tts.say("Kein Marker erkannt.")
61         self.onStopped(-1)

```

Um nach dem Scannen aller Felder wieder in eine normale stehende Haltung zurückzukehren, wurde eine zusätzliche Choreographie-Box mit dem Namen „**StandUp**“ verwendet. Diese sorgt dafür, dass der NAO-Roboter aus der „Crouch“-Position wieder sicher in den Standard-Standmodus wechselt. Dies ist besonders wichtig, um nach dem Scan-Prozess mit dem nächsten Spielschritt (z. B. Spielfigur bewegen) fortfahren zu können.

Im Folgenden wird der vollständige Python-Code gezeigt:


```
1 class MyClass(GeneratedClass):
2     def _init_(self):
3         GeneratedClass._init_(self, False)
4
5     def onLoad(self):
6         self.nTries = 0
7         self.postureService = self.session().service("ALRobotPosture")
8         pass
9
10    def onUnload(self):
11        self.postureService.stopMove()
12
13    def onInput_onStart(self):
14        if(self.nTries != self.getParameter("Maximum of tries")):
15            self.nTries = self.getParameter("Maximum of tries")
16            self.postureService.setMaxTryNumber(self.nTries)
17
18        result = self.postureService.goToPosture(self.getParameter("Name"),
self.getParameter("Speed (%)")/100.)
19        if(result):
20            self.success()
21        else:
22            self.failure()
23        pass
24
25    def onInput_onStop(self):
26        self.onUnload() #~ it is recommended to call onUnload of this box in a onStop method, as
the code written in onUnload is used to stop the box as well
27        pass
```

5.3 Beispielhafte Codeabschnitte mit Kommentierung

Beispiel 1: Siegesprüfung in `checkWinner()`

```
for row in self.grid:
    if row == ["X", "X", "X"]:
        self.tts.say("Du hast gewonnen!")
        self.onStopped()
        return True
```

Beispiel 2: Zugausgabe durch NAO

```
row, col = empty_cells[0]
self.grid[row][col] = "O"
feld_nummer = row * 3 + col
self.tts.say("Ich setze ein O auf Feld " + str(feld_nummer))
```

Beispiel 3: Markerinterpretation mit Fallback bei unbekannter ID

```
1 def interpret_marker(self, marker_id):
2     self.logger.info("interpret_marker() wurde mit ID {} aufgerufen".format(marker_id))
3
4     # Bekannte Marker-IDs → direkte Rückgabe
5     if str(marker_id) == "84":
6         return "leer"
7     elif str(marker_id) == "64":
8         return "X"
9     elif str(marker_id) == "112":
10        return "0"
11    else:
12        # Unbekannte ID → Sprachrückfrage
13        self.tts.say("Ich habe ein Problem, dieses Feld zu interpretieren. Ist es ein X, ein 0 oder leer hier?")
14        user_response = self.waitForUserInput() # hypothetisch definierte Eingabemethode
15        return user_response
16 |
```

Beispiel 4: Entlastung der Beinmotoren bei crouch position

```
# Roboter wartet auf Benutzereingabe am hinteren Tactil-Sensor, bevor der Marker gelesen wird
self.wait_for_rear_touch()

# Kritischer Schritt: Beinmotoren werden abgeschaltet, um Überhitzung zu vermeiden
motion.setStiffnesses("Legs", 0.0)

# Marker-Erkennung starten
self.marker_empfangen = False
self.onMarkRequest()
self.warteAufMarkerEmpfang()
```

Diese klar strukturierte Umsetzung in Python ermöglichte eine nachvollziehbare Spiellogik mit hoher Wiederholgenauigkeit in der Simulation.

6 Entwicklungsprobleme und Lösungen

6.1 Marker-ID-Probleme und Fehlinterpretationen

Während der Entwicklung stellte sich heraus, dass die Marker-ID-Erkennung im Realbetrieb mit dem physischen NAO-Roboter nicht zuverlässig funktionierte. Oft wurde die Marker-ID als -1 erkannt oder gar nicht. Diese Probleme traten insbesondere bei Markerpositionen am Rand des Sichtfelds oder bei leichten Neigungen des Roboters auf.

Ursachen:

Ungünstige Kameraperspektive (nicht zentriert auf den Marker)

Zu kleine Markergröße im Verhältnis zum Sichtfeld

Lichtreflexionen und Schattenwurf

Lösungsansätze:

Vergrößerung der Marker

Optimierung der Helligkeit, des Kontrasts und der Auflösung im Monitor (Kamera-Konfiguration)

Wiederholte Kalibrierung der Kopfbewegungen.

6.2 Einschränkungen bei der Kameraperspektive

Der NAO-Roboter verfügt über zwei Kameras: eine im Stirnbereich und eine im Mundbereich. Beide wurden genutzt, um verschiedene Felder des Spielfelds bestmöglich zu erfassen. Trotzdem war die Kameraposition häufig suboptimal, besonders bei Feldern in den unteren Ecken.

Lösungsansätze:

Testläufe mit beiden Kameras pro Feld

Reduktion des Betrachtungswinkels durch gezielte Kopfrotation

Priorisierung der Kamera mit dem besseren Sichtwinkel

6.3 Schwierigkeit der Bewegungskalibrierung

Die präzise Kopfbewegung über die neun Felder war essenziell für die Markererkennung. Kleine Abweichungen führten dazu, dass der Marker außerhalb des Kamerafokus lag. Diese Kalibrierung erwies sich als sehr zeitintensiv.

Lösungsansätze:

Manuelle Justierung jeder Kopfbewegung in Choregraphie

Reduzierung der Bewegungsgeschwindigkeit

Sequenzierte Einzelbewegungen statt durchgehender Rotation

6.4 Optimierung durch manuelle Testzyklen

Die genannten Herausforderungen konnten durch wiederholte Testreihen und gezielte manuelle Anpassungen schrittweise verbessert werden. Besonders hilfreich erwies sich dabei der Einsatz der Choregraphie-Simulation: In der virtuellen Umgebung traten keine Erkennungsprobleme bei Marker-IDs auf, da diese dort direkt und fehlerfrei simuliert wurden. Dies ermöglichte uns, die eigentliche Spiellogik und die Bewegungsabläufe des Roboters stabil zu entwickeln und unabhängig von Sensorproblemen zu testen. Im Gegensatz dazu führte der Einsatz des echten Roboters häufig zu Problemen bei der Erkennung der Marker-IDs – insbesondere dann, wenn die Kamera durch ungünstige Winkel, Lichtverhältnisse oder Bewegungsunschärfe keine gültige ID (z. B. 64, 84 oder 112) liefern konnte. Mithilfe der Simulation konnten wir somit gezielt eingrenzen, dass die Schwachstelle nicht in der Spiellogik, sondern in der realen Markeranalyse lag. Dieses Wissen half uns, gezielte Gegenmaßnahmen zu entwickeln – etwa die nachträgliche Nutzerabfrage bei unbekannten IDs.

7 Fazit und Ausblick

7.1 Bewertung der Projektergebnisse

Das Projektziel – die Entwicklung eines sprachgesteuerten Tic-Tac-Toe-Systems mithilfe des NAO-Roboters – wurde weitgehend erreicht. Die Kombination aus visueller Markererkennung, systematischer Kopfbewegung und verbaler Interaktion führte zu einem funktionalen Spielsystem. Besonders hervorzuheben ist die Stabilität des Programms in der Choregraphie-Simulation, wo alle Funktionen zuverlässig arbeiteten.

Im praktischen Einsatz mit dem realen Roboter traten jedoch – wie im vorherigen Kapitel beschrieben – Erkennungsprobleme auf, die vor allem auf die Hardwarebedingungen zurückzuführen sind. Diese Schwächen konnten den Gesamterfolg des Projekts jedoch nicht infrage stellen, da die Grundlogik und Systemstruktur tragfähig sind und durch manuelle Tests validiert wurden.

7.2 Reflexion des Entwicklungsprozesses

Das Projekt erforderte eine intensive Einarbeitung in die Roboterprogrammierung, insbesondere im Zusammenspiel zwischen visueller Analyse, Python-Code und Bewegungskontrolle. Die iterative Arbeitsweise, das ständige Testen in der Simulation und das Anpassen der Abläufe im Realbetrieb haben wesentlich zum Lernerfolg beigetragen.

Die klare Aufgabenverteilung im Team – von der Bewegungsentwicklung über das Gitterdesign bis hin zur Python-Implementierung – ermöglichte eine zielgerichtete Umsetzung.

7.3 Erweiterungsmöglichkeiten (z. B. Spracherkennung, AI-Gegner)

Das aktuelle Projekt stellt eine solide Basis dar, die in verschiedene Richtungen erweitert werden kann:

Spracherkennung: Durch Integration eines aktiven Sprachmoduls könnte der Benutzer seine Züge ansagen, was die Interaktion noch intuitiver macht.

Visuelles Tracking: Verbesserung der Markerpositionierung und -erkennung durch 3D-Kalibrierung oder Alternativen zu NAOMarkern

KI-Gegner: Implementierung einer echten KI-Strategie zur Entscheidungsfindung bei Spielzügen statt nur Setzen auf das erste freie Feld

Multiplayer-Modus: Ermöglichung eines Spiels zwischen zwei Personen mit NAO als Schiedsrichter

Insgesamt bietet das System eine gute Grundlage für weitere Forschung im Bereich Mensch-Roboter-Interaktion und KI-gesteuertes Verhalten.

8 Literaturverzeichnis

- [1] SoftBank Robotics. „NAO Robot Overview.“ Online verfügbar:
<https://www.softbankrobotics.com/emea/en/nao>
- [2] Aldebaran Robotics (Archiviert). „NAO Documentation Center.“ Zugriff über: <http://doc.aldebaran.com/>
- [3] NAOqi API Dokumentation. SoftBank Robotics Developer Center.
<https://developer.softbankrobotics.com/naoqi>
- [4] Choregraphe Suite – Benutzerhandbuch. SoftBank Robotics, 2018.
- [5] Python.org – Offizielle Website zur Python-Programmiersprache. <https://www.python.org>
- [6] OpenCV Dokumentation zur Bildverarbeitung. <https://docs.opencv.org/>
- [7] Prof. Dr. rer. nat. Michael Böhmer. „Robotik-Vorlesungsskript: Sensorik und Aktorik“, Hochschule Bochum, 2023.
- [8] NAOqi-Vision Module: ALVisionRecognition – Entwicklerdokumentation. Zugriff:
<https://developer.softbankrobotics.com/naoqi/vision/alvisionrecognition>
- [9] Choregraphe-Tutorials (YouTube): Einstieg in NAO-Programmierung. Autor: Robotics Lab Bochum.
- [10] Dipl.-Ing. (FH) Oliver Mathew. „Technische Mensch-Maschine-Interaktion (TMMI) – Vorlesungsskript“, Hochschule Bochum, 2024.
- [11] Eigenes Projektmaterial, Teamaufzeichnungen, Code und Markerdesign (2025).